

PinFM: Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform

Xiangyi Chen
Pinterest
San Francisco, CA, USA
xiangyichen@pinterest.com

Kousik Rajesh
Pinterest
San Francisco, CA, USA
krajesh@pinterest.com

Matthew Lawhon
Pinterest
San Francisco, CA, USA
mlawhon@pinterest.com

Zelun Wang
Pinterest
San Francisco, CA, USA
zelunwang@pinterest.com

Hanyu Li
Pinterest
San Francisco, CA, USA
hanyuharryli@pinterest.com

Haomiao Li*
Pinterest
San Francisco, CA, USA
haomiaoli@pinterest.com

Saurabh Vishwas Joshi
Pinterest
San Francisco, CA, USA
sjoshi@pinterest.com

Pong Eksombatchai
Pinterest
San Francisco, CA, USA
pong@pinterest.com

Jaewon Yang
Pinterest
San Francisco, CA, USA
jaewonyang@pinterest.com

Yi-Ping Hsu*
Pinterest
San Francisco, CA, USA
yhsu@pinterest.com

Jiajing Xu
Pinterest
San Francisco, CA, USA
jiajing@pinterest.com

Charles Rosenberg
Pinterest
San Francisco, CA, USA
crosenberg@pinterest.com

Abstract

User activity sequences have emerged as one of the most important signals in recommender systems. We present a foundational model, PinFM, for understanding user activity sequences across multiple applications at a billion-scale visual discovery platform. We pretrain a transformer model with 20B+ parameters using extensive user activity data, then fine-tune it for specific applications, efficiently coupling it with existing models. While this pretraining-and-fine-tuning approach has been popular in other domains, such as Vision and NLP, its application in industrial recommender systems presents numerous challenges. The foundational model must be scalable enough to score millions of items every second while meeting tight cost and latency constraints imposed by these systems,. Additionally, it should capture the interactions between user activities and other features and handle new items that were not present during the pretraining stage.

We developed innovative techniques to address these challenges. Our infrastructure and algorithmic optimizations, such as the Deduplicated Cross-Attention Transformer (DCAT), improved our throughput by 600% on Pinterest internal data. We demonstrate that PinFM can learn interactions between user sequences and candidate items by altering input sequences, leading to a 20% increase in engagement with new items. PinFM is now deployed to help improve the experience of more than a half billion users across various applications.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

Sequential Recommendation, Recommendation System

ACM Reference Format:
Xiangyi Chen, Kousik Rajesh, Matthew Lawhon, Zelun Wang, Hanyu Li, Haomiao Li, Saurabh Vishwas Joshi, Pong Eksombatchai, Jaewon Yang, Yi-Ping Hsu, Jiajing Xu, and Charles Rosenberg. 2025. PinFM: Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25), September 22–26, 2025, Prague, Czech Republic.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3705328.3748050

1 Introduction

Recommender systems (RecSys) provide relevant and engaging content to users in many social media platforms. For personalization, recommender systems need to comprehend users' past interaction histories, often referred to as user activity sequences. Early methods focused on extracting the attributes of the items users interacted with [22]. More recent approaches have recognized the importance of the sequential nature of user activities and are therefore focused on developing model architectures specialized for sequences, such as transformers. Recent studies have also found that utilizing a large model to understand user activity histories enhances predictive performance and user engagement metrics [30, 31].

*Work done at Pinterest.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '25, Prague, Czech Republic

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1364-4/25/09
https://doi.org/10.1145/3705328.3748050

In this paper, we aim to build a foundational model for understanding user activity sequences for multiple RecSys applications at a billion-scale visual discovery platform. We pretrain a large foundational model with a massive amount of user activity sequence data collected across multiple applications. For each downstream application, we fine-tune the foundational model with application-specific training data and application-specific features. This approach allows for training a very large model efficiently because the pretraining can be done for multiple applications. It also allows for integrating the foundation model easily with existing application model. For these reasons, this pretraining-and-fine-tuning approach has become highly popular in both large language models and computer vision tasks.

There are many challenges in building large foundational model for understanding user sequences for multiple recommendation applications at industrial scale. First, since the foundational model needs to be fine-tuned with a downstream recommender system model, it is crucial to keep the training and serving cost and latency at a manageable level. Recommender system models need to process the request within hundreds of milliseconds and score millions of items every second [26]. Developing a large model to satisfy this latency budget is very challenging. Second, applications would have different sequences and application-specific features about the candidate item and the user. It is crucial to capture the interaction between the user activities and other features used in the application [32]. Third, foundational models should generalize to new items not present in the pretraining data, as users keep introducing new items everyday.

Though the effectiveness of large sequence models is well known in industrial recommender systems [21, 30], to our knowledge, there is no existing literature regarding developing foundational models for user sequences to be integrated with multiple downstream recommender system models. Instead, large sequence models such as HSTU [30], TIGER [21], and TWIN-V2 [23] are often used as standalone models trained for specific applications, which can become prohibitively expensive if applied across many applications. An alternative approach to developing large foundational models is to use them as teacher models in knowledge distillation [12, 15]. Unfortunately, this method significantly slows down experimentation, as it requires waiting for the production model to learn from the new teacher model, a process that can take months [12].

We present PinFM, a foundational model for user activity understanding. We pretrain a large scale transformer using a massive dataset of user activity sequences. In particular, we pretrain a 20 billion parameter model with 2 years of user activity history. To expedite pretraining, we avoid using high-dimensional features for each user activity, such as pretrained item embeddings. Instead, we rely on categorical features like item IDs and action types, learning embeddings for each ID. Once pretraining is complete, we integrate the pretrained transformer and embedding tables within the recommender system models for multiple applications. Each downstream ranking model uses this pretrained transformer and embedding table to encode user activity sequences and combine it with other existing features. To address the first challenge of training / serving cost, we developed some novel optimizations technique such as the Deduplicated Cross-Attention Transformer (DCAT) and int4 quantized embeddings. These improvements allow us to deploy PinFM to the two most important recommender systems, the Home Feed and Related Items Feed, with neutral cost and marginal latency increases. To address the second challenge of capturing interactions between the user sequence and other features, we explored different ways of creating the input sequence for the foundational transformer, so that the transformer can capture interactions via cross attention. To address the last challenge of handling unseen items, we added modeling techniques such as leveraging item features and additional drop out for fresh items.

The summary of our contributions are as follows:

- **Foundational paradigm for user activity understanding**: We propose a pretraining-finetuning approach for user activity sequence models. Our pretraining method can handle years of user activity sequences efficiently by avoiding high dimensional features. The pretrained model can be easily integrated with multiple downstream models, as user activity sequences are a common signal for most recommender systems. To the best of our knowledge, this is the first foundational model for understanding user sequences in the industry.
- **Efficiency improvements for cost-neutral integration with downstream models**: We developed several efficient computational techniques for PinFM. For instance, PinFM processes the user activity sequence just once, reusing the results for all candidates during the ranking stage. Additionally, we implement aggressive quantization for the large embedding tables. These techniques allow PinFM to be seamlessly integrated into a wide range of downstream recommender models. This integration does not affect latency or serving costs, allowing downstream models to continue utilizing other features as they normally would.
- **Input sequence engineering for learning user-candidate interactions:** We conducted experiments to demonstrate that PinFM can capture the interactions between the user activity sequences and other features, in particular candidate item features. This flexibility enables us to design input sequences tailored to different applications to meet specific needs. For example, in the Home feed application, where there are many fresh items, we incorporated item features into the input sequence to address the cold start problem. This adaptability is crucial for making a significant impact across multiple applications.
- **Sharing lessons from live traffic experiments:** We deployed PinFM to the platform's two most crucial recommender systems, the Home feed and the Related Items feed. Through extensive ablation studies conducted in both offline and online tests, we identified the most impactful techniques.

The rest of paper is organized as follows. Sec. 2 discusses related work. Sec. 3 describes PinFM, and Sec. 4 presents infrastructure optimization. Sec. 5 presents offline and online experimental results. We conclude in Sec. 6.

2 Related Work

We discuss related work in two areas: User sequence understanding models and Foundation model for recommender systems.

User sequence models: Users' past interaction histories have been valuable signals for Recommender systems from early days [22]. Transformers, by capturing the interactions in sequential data via self-attention, revolutionized user sequence understanding models [11, 19, 23, 26]. One approach is to use sequence models to predict the next item that the user would interact with. These methods have shown promising results in academic benchmarks [6, 7, 16] and in industry [21, 30]. However, they can only handle sequences; it is not straightforward for them to leverage features that are designed for non-sequential models like DCN or DLRM [17, 24].

Another line of methods is developing a sequence encoder within an existing recommender system models. Methods like TransAct [26], TWIN [2, 23] developed a transformer to encode the user activity sequences. These transformers are trained with the downstream application models with other application-specific features. We aim to combine both approaches. We pretrain a sequence-only model to train user activities across applications, but we fine-tune PinFM with existing features in the downstream models.

Foundation models for recommender systems Foundation models for recommender systems are large models trained across various applications to handle multiple recommendation tasks. For a detailed survey, please refer to [8]. One major trend is to use large language models as a backbone for a foundation model. Unis4Rec [7], RecFormer [14] and PTUM [25] showed promises in academic benchmarks, and M6Rec [3] and 360Brew [5] did in industrial applications. Despite the impressive offline results, these models are computationally heavy due to processing long prompts that describe the full feature vector in text. Therefore, we decided to develop a foundation model that encodes user sequences using a sequence of item IDs, which is significantly more compact than text.

3 Methodology

As mentioned earlier, our framework contains two stages, pretraining a user sequence model and fine-tuning with downstream ranking model, an illustration of the two-stage paradigm can be found in Figure 1. In this section we explain our method in more details.

3.1 Pretraining

In pretraining stage, we collect user sequence data and use a few variants of next token prediction loss commonly used in language models. In our framework, the backbone model architecture can be any transformer-based decoder only models and we choose the GPT2 architecture [20] with Pre-LN [28] in all our experiments to balance performance and latency requirements. We also tried HSTU [30] architecture and got similar results with GPT2. In the next, we discuss how we construct data and modified loss functions used in large language models to address the challenges unique to recommender systems.

Data Construction. We collect each user's historical actions on the platform and sort them in ascending order by timestamp, capping the maximum sequence length at 16,000. We denote a user's full action sequence as $\mathbf{S} = [S_1, S_2, \ldots, S_N]$, where each element S_i represents the i-th event. Specifically, $S_i = [t_i, a_i, v_i, id_i]$ is a tuple consisting of the timestamp t_i, the action type a_i, surface type v_i, and the identifier id_i of the item on which the action was performed. During training, we cut the user sequences into non-overlapping segments of length L with L being a few hundred.

Modeling and Training Objectives. In training, given a possibly truncated user sequence $\mathbf{S}$ with length m, we map each item identifier id_i to an embedding vector $\mathbf{E}_i = \text{emb}(id_i)$, forming the embedding matrix $\mathbf{E} = [\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_m]$. The surface and action type sequences are also converted to embedding matrices $\mathbf{V} = [\mathbf{V}_1, \mathbf{V}_2, \ldots, \mathbf{V}_m]$ and $\mathbf{A} = [\mathbf{A}_1, \mathbf{A}_2, \ldots, \mathbf{A}_m]$ by an embedding table. Next, we apply a pointwise MLP with an l_2 normalization layer, denoted ϕ_{in}, to produce an input to the backbone model M. The final hidden states from M are then passed through another pointwise MLP with normalization, ϕ_{out}, yielding the user representation sequence $\mathbf{H}$. Formally speaking,

$$\mathbf{H} = [\mathbf{H}_1, \mathbf{H}_2, \ldots, \mathbf{H}_m] = \phi_{\text{out}}(M(\phi_{\text{in}}(\mathbf{E} + \mathbf{V} + \mathbf{A}))) \tag{1}$$

For our training objectives, we build upon language model pretraining [1] and introduce novel objectives specifically designed for recommender systems. We begin with a commonly used loss function for user sequence modeling—the next token prediction loss, which is predicting the next item based on the user's activities leading up to that point. However, previous studies [9, 19] have shown that next token prediction alone is insufficient to capture the evolution of users' interests over time. Since users can have multiple interests and switch between short-term and long-term interests, it is beneficial to aim at predicting multiple tokens simultaneously (see Sec. 5.1 for more details). To address this, we incorporate multiple types of future token predictions during pretraining.

There are additional differences between pretraining user activity models and language models. In user activities, we have action types, with varying levels of importance among them. Additionally, the vocabulary size for user activities—the number of unique items—can reach billions, making it computationally expensive to calculate the prediction probability for every token in the vocabulary. To address these challenges, we formulate our pretraining objectives as follows.

1. We adopt infoNCE-based loss [18] due to large vocabulary size. For each user representation $\mathbf{H}_i$, when there is a positively engaged item after ith position in the sequence, whose embedding is obtained by $z_{i+1} = \psi(\text{emb}(id_{i+1}))$ with ψ being another MLP followed by an l_2 normalization layer, the loss function below can be applied

$$l(\mathbf{H}_i, z_{i+1}) = -\log \frac{\exp(sim(\mathbf{H}_i, z_{i+1})/\tau)}{\exp(sim(\mathbf{H}_i, z_{i+1})/\tau) + \sum_{k=1}^{K} \exp(sim(\mathbf{H}_i, z_k^-)/\tau)} \tag{2}$$

where we choose $sim(a, b) = a^T b$ being the inner product, τ being the temperature parameter and we make it learnable with a small initial value. z_k^- are K embeddings sampled in-batch excluding items positively engaged by the same user, these are treated as negative samples in the contrastive loss.

2. As indicated in equation 2, the loss is only applied when there is a future positively engaged item for $\mathbf{H_i}$. This can drastically reduce computation compared with predicting tokens at every position. Let A_{pos} denotes the set of predefined positive actions and $\mathbb{1}[]$ being the indicator function, the Next Token Loss can be defined

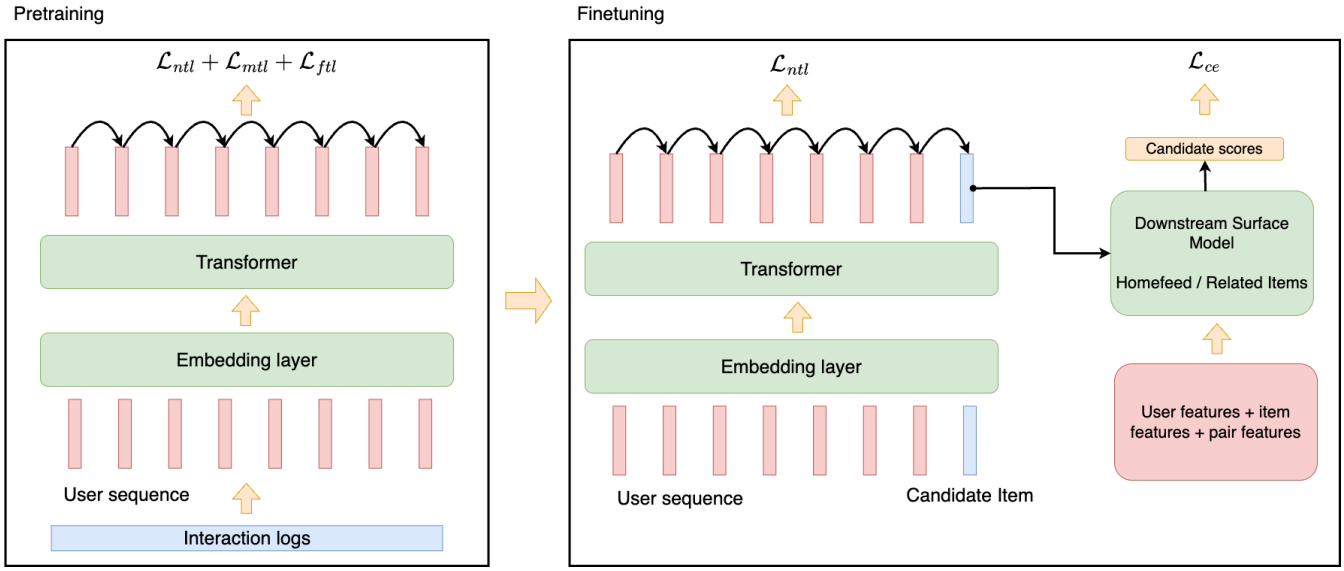


Figure 1: Training and fine-tuning of PinFM

below

$$\mathcal{L}_{ntl} = \sum_{i=1}^{m-1} l(\mathbf{H}_i, z_{i+1}) \mathbb{1}[a_{i+1} \in A_{pos}]$$

3. In addition to predicting each positively engaged next token, we also want the model to predict future tokens in a window of length L' since user interests tend to be consistent in a short period of time. Thus, we add an extra Multi Token Loss defined as below.

$$\mathcal{L}_{mtl} = \sum_{i=1}^{m-1} \sum_{j=i+1}^{i+L'} l(\mathbf{H}_i, z_j) \mathbb{1}[a_j \in A_{pos}]$$

where z_j is a positive item within the window.

4. Our model will eventually be used in downstream ranking models, and these ranking models often use a real-time sequence with max length L_d smaller than L due to latency constraints. The losses $\mathcal{L}_{ntl}$ and $\mathcal{L}_{mtl}$ put approximately equal weights on predicting future tokens for each input sequence length, but we want the model to be more predictive especially when sequence length is close to downstream input sequence length L_d. To achieve this, we add another loss to predict a future window of positive tokens on $\mathbf{H}_{L_d}$, we term this the Future-Token Loss ($\mathcal{L}_{ftl}$).

$$\mathcal{L}_{ftl} = \sum_{j=n+1}^{n+L'} l(\mathbf{H}_{L_d}, z_j) \mathbb{1}[a_j \in A_{pos}]$$

This is similar to instruction fine-tuning in large language models, where we present a task to the model by providing a complete input sequence (instruction) and prompting it to generate the output.

3.2 Fine-tuning

Once the model is pretrained on the user sequence data, we fine-tune it with downstream ranking models. In the ranking stage, capturing the interaction between user features (including PinFM) and candidate item features is crucial. We will discuss how to achieve this without significantly increasing computational costs.

Ranking model integration. Most of our ranking models are classification models with feature crossing layers like DLRM [17] or DCN [24]. In a request for a specific user, we rank the candidate items returned by retrieval models. For each candidate item, the ranking model applies modules that process user features, candidate item features, and interaction features. The outputs of these modules are then aggregated to predict the probabilities of various user actions for the candidate item. An example of ranking model can be found in [27] with more details.

When integrating with ranking models, we add the pretrained model as a module for the user activity sequence. One key difference compared to pretraining comes from the presence of the candidate item. There are two ways to fuse the user activity sequence and the candidate item.

1. Late fusion. The module can take user sequence as input to produce a user embedding, which will be aggregate with outputs from other modules later. This setting makes the input sequence to PinFM identical to the pretraining task because we do not consider the candidate item. We can easily cache the output of PinFM for candidate items related to the same request because the user activity sequence remains identical. However, the downside is that PinFM won't be able to contextualize the user activity sequence for each candidate item.

2. Early fusion. The candidate item is appended to the user sequence used as input of the user sequence module. In this case, we can output a user embedding and another embedding crossing the user information with candidate information [30]. Due to its stronger prediction ability (See Sec. 5.1), we choose this way. With this approach, the input sequence to PinFM varies for each candidate item within the same request, but we will show in Sec 4.1 that we can still efficiently cache intermediate results for the candidates in the same request.

It is noteworthy to mention that for the candidates to be ranked, there are typically some extra embedding features available, such as content embedding, graph-based embedding. For an extra candidate

embedding, we also optionally project and sum it with the projected candidate id embedding as the user sequence module input, an extra loss to align the projected embedding into the user sequence module input space is added in this case.

To let the pretrained model adapt to the sequence distribution of downstream ranking models faster, we can optionally add the losses $\mathcal{L}_{ntl}$ and $\mathcal{L}_{mtl}$ in the fine-tuning stage in addition to the downstream ranking model losses. Also, we set the learning rate of the pretrained module to be around 1/10 of the ranking model learning rate to keep the pretrained information from overwritten.

To further align the learning target of the user sequence module with the downstream model, we also apply the ranking losses of the downstream model to the user sequence module outputs, together with an MSE loss to align the predictions given by our user sequence module and the final ranking model predictions. We found this help boost the performance of downstream ranking models.

Handling Cold-start items. As it is commonly known, id embeddings only perform well for items with enough interaction history. When we add a module that is strongly dependent on the candidate item id embedding, the model performance on cold-start items could gets hurt. To recover the performance, we found techniques below to be helpful

1. Candidate item id randomization. During fine-tuning, we randomly choose an id for the candidate item 10% of times. This can simulate cold-start situations since the cold-start item ids are randomly generated and not seen by the model during training.

2. Dropout on module outputs for fresh candidate items. During fine-tuning, we add dropout to the outputs of our pretrained module if the candidate item is fresh, i.e., generated within T days by the training sample request time. With such change, the model can hopefully rely less on outputs of the module that are derived from id embeddings for fresh candidates.

More details on these techniques along with others that helped cold-start performance can be found in Sec. 5.1.

3.3 Discussion on design choices

Unidirectional vs Bidirectional modeling. During pretraining we prefer unidirectional transformers for similar reasons as large language models. Compared to bidirectional transformers, unidirectional models trained using next token prediction objectives hold more predictive power, allowing the model to generalize to new user sequences. In fine-tuning, if we eliminate causal modeling in favor of bidirectional attention we note a 1.4% drop in evaluation metrics. We hypothesize that this drop is due to the distribution shift between pretraining and fine-tuning. Further, the use of causal attention allows us to significantly optimize the attention mechanism for data patterns we observe as explained in section 4.1.

ID vs pretrained embeddings in sequence. Instead of using ID-based sequence representation, a straightforward alternative is to use pretrained content-based or engagement-based embeddings. However, using pretrained embeddings will put more pressure and challenges on infrastructure since the data volumn for both storage and serving needs to be hundreds of times larger for the user sequence features. Therefore, we choose ID-based representation for its flexibility and scalability.

4 Efficiency

The framework's core components are transformers and large embedding tables. We allocated significant engineering resources to optimize both components, as their efficiency is critical for the project's feasibility from an infrastructure cost standpoint.

4.1 Deduplicated Cross-Attention Transformer (DCAT)

To deploy out large transformer architecture in online ranking systems handling a few million queries per second, we capitalize on a key data pattern. There are significantly fewer number of unique user sequences compared to the number of candidates that are scored – the ratio is 1:1000 during serving and 1:10 during training. The transformer computation in our model architecture can be separated into two components: (1) the context component, which involves applying the transformer to a user's historical action sequence and (2) the crossing component, where each item to be scored is evaluated in conjunction with the user history. The context component is computed only once per user and the keys and values of each transformer layer are stored as KV cache. We then use the context KV cache to perform cross-attention with the entire list of candidate items. This optimization is implemented in both the training and serving phases with custom kernels using OpenAI Triton, and helped us achieve a 600% increase in serving throughput and 200% increase in training throughput over regular self attention implemented with FlashAttention [4].

Now we explain DCAT in more details. We start with context component, which produces transformer KV cache for the crossing component using de-duplicated user sequence. Let $\mathbf{X}^{(0)} \in \mathbb{R}^{B \times L_d \times d}$ be a batch of user sequence embeddings as input of the transformer, and Ψ be an invertible deduplication operation in the batch dimension. We first deduplicate the sequence input by

$$\mathbf{X}_u^{(0)} = \Psi(\mathbf{X}^{(0)}) \in \mathbb{R}^{B_u \times L_d \times d}$$

and $\mathbf{X}_u^{(0)}$ is the deduplicated batch, typically $B_U = B/16$. Denote $\mathbf{X}_u^{(l-1)}$ as the input to l-th layer of the transformer, then we can compute $\mathbf{X}_u^{(l)}$ by

$$\begin{gathered}\mathbf{Q}_u^{(l)} = W_q X_u^{(l-1)}, \quad \mathbf{K}_u^{(l)} = W_k X_u^{(l-1)}, \quad \mathbf{V}_u^{(l)} = W_v X_u^{(l-1)} \\ \mathbf{X}_u^{(l)} = g(\text{Attention}(\mathbf{Q}_u^{(l)}, \mathbf{K}_u^{(l)}, \mathbf{V}_u^{(l)}), X_u^{(l-1)})\end{gathered} \tag{3}$$

with $g()$ being some processing function that are irrelevant to the context here and Attention() being the standard softmax attention. We save $\mathbf{K}_u^{(l)}$ and $\mathbf{V}_u^{(l)}$ as KV cache to be used later. In the crossing component, we have a batch of candidate embeddings $\mathbf{X}_c^{(0)} \in \mathbb{R}^{B \times 1 \times d}$ as transformer input. For each layer, we apply the inverse of the deduplication operation Ψ^{-1} to the KV cache and concatenate them with candidate KV, before performing cross attention. Given $\mathbf{X}_c^{(l-1)}$ as l-th layer input, $\mathbf{X}_c^{(l)}$ can be written as

$$\begin{gathered}\mathbf{Q}_c^{(l)} = W_q X_c^{(l-1)}, \quad \mathbf{K}_c^{(l)} = W_k X_c^{(l-1)}, \quad \mathbf{V}_c^{(l)} = W_v X_c^{(l-1)} \\ \mathbf{X}_c^{(l)} = g(\text{Attention}(\mathbf{Q}_c^{(l)}, \Psi^{-1}(\mathbf{K}_u^{(l)}) \,||\, \mathbf{K}_c^{(l)}, \Psi^{-1}(\mathbf{V}_u^{(l)}) \,||\, \mathbf{V}_c^{(l)}), X_c^{(l-1)})\end{gathered} \tag{4}$$

and $||$ operator denotes concatenation along the sequence dimension.

In training, Ψ is implemented using Pytorch and in serving the inference server uses pointers to perform the same deduplication. In both training and serving, Ψ^{-1} is implemented using a Triton kernel and serves as a "broadcasting" operation that uses the unique sequence indices from $\mathbf{X}^{(0)}$ to invert the deduplication operation.

GPUs typically perform much better when input tensors are aligned to powers of 2. We further optimize DCAT by maintaining the sequence length to be 256 at all times, eliminating all concatenations, and instead replacing the oldest tokens in KV cache with the candidate KV and rotating the attention mask. During serving, we also skip the self attention operation associated with the calculation of $X_u^{(l)}$ for the last layer, since this is only used for loss computation. Together, these increase our throughput by 25% additionally over the 600% improvement from DCAT.

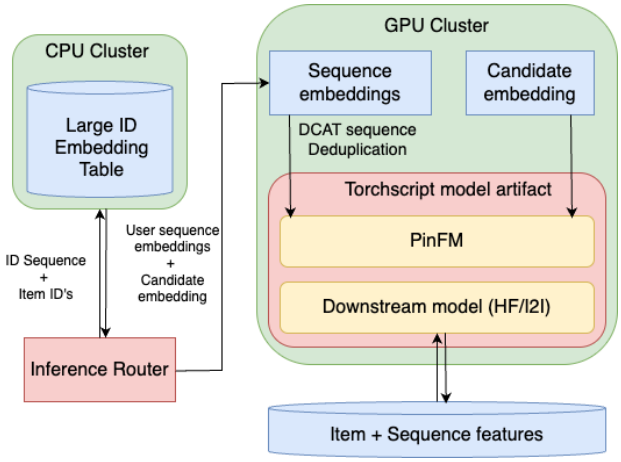


Figure 2: Serving Infrastructure for PinFM

4.2 Large Embedding Table

We adopted a straightforward setup for both the training and serving of our large embedding tables. On the training side, we utilized TorchRec [10] to distribute our tables across multiple GPUs. On the serving side, the tables are hosted on a single CPU host, separate from the GPU inference host, as its size is manageable within the memory limits of a single CPU. While the training setup is scalable, the serving setup is less flexible due to the memory constraints of each CPU host and the significant communication costs incurred when the sequence of embeddings are transferred to the GPU inference host. To address this, we investigated various compression techniques to decrease data size, ultimately adopting a post-training quantization strategy.

Embedding quantization: Within the proposed foundation model, the vast majority of the parameter increase is attributed to the large embedding table. For example, in our standard setup, for each item id id_i, we obtain a 256 dim float16 embedding vector through looking up 8 sub embedding tables each with 80 million rows and 32 dim, which helps mitigate hash collision: $\mathbf{E}_i = \text{emb}(id_i) = \bigotimes_{j=0}^{7} \text{emb}_j(\text{hash}_j(id_i))$ in which we use $\bigotimes$ to denote concatenation of the the 8 sub embeddings. The embedding tables overall adds 20 billion trainable parameters. While vastly increasing the model capacity, it also poses a significant challenge for serving.

We therefore performed quantization after training the model. Specifically, we used FBGEMM[13]'s n-bit min-max quantization kernels as backbone and performed post-training quantization (PTQ) across all shards of the embedding table, effectively converting each 32-dim fp16 vector to 32 int8/int4 + 1 fp16 scale value + 1 fp16 bias value, and then perform bitpacking to fp32 to reuse torch native embedding API. In an int4 setting, this compresses each vector from 512 bit to 160 bit, and reduces the embedding table to 31.25% of its original size and cuts CPU host cost proportionally.

At serving time, we serve the compressed table on CPU host as in Figure 2 while on GPU, we optimize decoding by using a custom Triton kernel that combines bit unpacking and FBGEMM dequantization. In online serving test, we observed negligible GPU forward pass latency overhead and overall inference API call latency was reduced by 7% due to lowered embedding IO load.

We quantify the effect of quantization by calculating the L2 norm of the value deviation between original embedding and quantized embedding vs the L2 norm of original embedding, we observed 0.45% at int8 quantization, and 7.8% at int4.

In offline evaluation of Save metrics, we observed that int8 quantization had negligible effect while int4 brings 0.06% drop versus full precision model, which is still quite minor. We proceeded with an online A/B test with int4 PTQ against full precision model. Over two weeks, we observed statistically neutral performance and validated int4 quantization as a viable optimization.

4.3 Serving Infrastructure

Figure 2 shows the serving infrastructure design for PinFM. Notably, we separate our large ID embedding table from the dense model and serve it on a CPU cluster. This allows us to scale our embedding table and model size without having to share the limited GPU memory available during inference. The inference router is responsible for fetching the relevant ID embeddings for the user sequence and candidate item from the CPU cluster and passing it to the GPU cluster for inference. The sequence embedding is de-duplicated here before model forward using DCAT. The final token output from PinFM is then passed to the downstream model for score prediction.

5 Experiments

In this section, we compare PinFM to baselines, evaluate different designs for PinFM, and explore the optimal input sequence engineering for downstream applications. We also show that PinFM delivers strong metric wins in online A/B tests in multiple applications.

5.1 Offline Experiments

For offline experiments, we modify PinFM and pretrain it for different ablations. We then fine-tune with downstream applications, and report the downstream application metrics.

Dataset and metrics: As described in Sec 3, we use a pretraining dataset of last 2 years of user activities. For fine-tuning applications, we use the the platform's Home Feed (HF) [27] and Related Items (Item2Item, I2I). When a user comes in to the platform, the Home Feed recommends the most relevant and engaging items. When the user clicks on a particular item, I2I shows other items that are

related to the clicked item. Both recommender systems consist of retrieval models and ranking models, and we apply PinFM to the ranking models. For fine-tuning data, we use 3 weeks of HF / I2I data.

HF and I2I are the two most important ranking models for user engagement at the platform. Both models are point-wise multi-task learning models to predict probabilities of various user actions, like Save (save to users' collections), Click, Share, Hide, etc. Both models use item features, user features and context features as input, and both surfaces aim to recommend personalized feed and to serve fresh content. The difference between HF and I2I is that HF recommendations are diverse and exploratory, whereas I2I has an emphasis on relevance between candidate items and the query item and focus more on recommending content that is similar to the query item.

For evaluation, we use metric that we call HIT@3. The HIT@3 metric evaluates the performance of a ranking model by assessing whether the top three recommended items in a group of items recommended at the same time received corresponding user action, e.g., HIT@3 of Save evaluates on average how many top three recommended items by the model received Save action. A higher HIT@3 score indicates better model performance for positive actions, whereas a lower score is preferable for negative actions like Hide, as described in [27]. In all the results, we report relative lift in HIT@3 despite the action being positive or negative.

Input sequence construction during fine-tuning: There are different ways to construct the input sequence being passed into the PinFM transformer and to extract output embeddings which are then used in feature crossing. Specifically, we compare the following variations with the HF ranking model without PinFM:

- PinFM-base: take the transformer output for the candidate item and the pretrained embedding for the candidate item, use these two embeddings for feature crossing. This is the exact version described in Figure 1
- PinFM-GraphSAGE: on top of the PinFM-base, sum candidate GraphSAGE [29] embedding with the pretrained id embedding as the representation of the candidate item in transformer input.
- PinFM-GraphSAGE-LT: on top of PinFM-GraphSAGE, add a learnable token to the sequence before candidate embedding. Use the transformer output of the candidate item, the learnable token and the pretrained embedding for the candidate item for feature crossing.
- PinFM-lite-mean: without inserting candidate item into sequence, simply mean pool the user sequence output and add this embedding along with the candidate item embedding as input and cross with other input features later in the model.
- PinFM-lite-last: in PinFM-lite-mean, replace mean pooling with selecting the last token.

The comparison of aforementioned variants on HF and I2I ranking models is shown in Table 1. We can see a clear trade-off between metric improvements and complexity of the model. PinFM-GraphSAGE-LT performs the best and it is also the most complex variant. On contrary, PinFM-lite groups have least metric gains but they require no inclusion of the candidate item in transformer and thus benefit from vastly improved serving efficiency.

Table 1: Percentage change in fine-tuning ranking model Save HIT@3 with different input sequence.

Input seq	HF	I2I
w/o PinFM	0.0%	0.0%
PinFM-base	2.91%	1.76%
PinFM-GraphSAGE	3.08%	1.92%
PinFM-GraphSAGE-LT	3.76%	1.92%
PinFM-lite-mean	1.87%	1.53%
PinFM-lite-last	1.93%	1.49%

Candidate item representation in fine-tuning for cold start: To improve cold start performance of candidate items with few engagements, we find a variety of regularization and special representation techniques to be important. We compare the effectivenes of these techniques on HF ranking model in Table 2 and different variants are detailed below:

- PinFM-cs-none: the basic version of PinFM with no special handling of item freshness in PinFM.
- PinFM-cs-CIR: PinFM-cs-none with Candidate Item Randomization (CIR), meaning 10% of candidate item ids are randomized during training before entering embedding lookup.
- PinFM-cs-CIR-IDD: PinFM-cs-CPR with Item-age Dependent Dropout (IDD) where we add item age dependent dropout. More specifically, a dropout layer with $p = 0.7$ is added to PinFM produced embeddings when candidate item age is less than 7 days, and a dropout layer with $p = 0.5$ is applied to PinFM produced embeddings when item age is between 28 days and 7 days.
- PinFM-cs-CIR-IDD-GSLT: PinFM-cs-CIR-IDD with summing candidate GraphSAGE and addding a learnable token (GSLT) in sequence, this become the same as PinFM-GraphSAGE-LT in Table 1.

To measure the cold start performance, we measure the HIT@3 for Saves in HomeFeed toward the items that are fresher than 28 days old (HF 28d) and than 7 days old (HF 7d). We can see that the techniques help significantly, turning HF 28d Save metric from around -4% to 17%, achieving more than 20% boost.

Table 2: Percentage change in HF ranking model Save HIT@3 with different cold start techniques.

Candidate Item Repr.	HF Overall	HF 28d	HF 7d
w/o PinFM	0.0%	0.0%	0.0%
PinFM-cs-none	3.36%	-4.40%	-23.98%
PinFM-cs-CIR	3.43%	1.25%	-4.38%
PinFM-cs-CIR-IDD	3.49%	10.71%	8.16%
PinFM-cs-CIR-IDD-GSLT	3.76%	17.72%	12.01%

Pretraining and fine-tuning losses: Model performance of different setting of pretraining and fine-tuning losses evaluated on HF ranking model is shown in Table 3. It can be seen that Save metric keep increasing as we add more losses, Hide metric improves when $\mathcal{L}_{mtl}$ is added but deteriorates when $\mathcal{L}_{ftl}$ is added.

The effects of the sequence losses in fine-tuning are also shown in Table 3. We can see that removing $\mathcal{L}_{ntl}$ will lead to a sizable

drop in Save with the benefit of decreasing Hide, while further adding $\mathcal{L}_{mtl}$ can provide tiny increase in Save and recover Hide. However, adding $\mathcal{L}_{mtl}$ will further increase GPU memory usage and slowdown training and thus, we do not include $\mathcal{L}_{mtl}$ by default in fine-tuning.

Table 3: Percentage change in HIT@3 when changing losses, baseline is using $\mathcal{L}_{ntl}$ for both pretraining and fine-tuning.

pretraining Loss	fine-tuning Loss	Save (lift)	Hide (lift)
$\mathcal{L}_{ntl}$	$\mathcal{L}_{ntl}$	0.0%	0.0%
$\mathcal{L}_{ntl} + \mathcal{L}_{mtl}$	$\mathcal{L}_{ntl}$	0.42%	-1.27%
$\mathcal{L}_{ntl} + \mathcal{L}_{mtl} + \mathcal{L}_{ftl}$	$\mathcal{L}_{ntl}$	0.95%	2.43%
$\mathcal{L}_{ntl} + \mathcal{L}_{mtl} + \mathcal{L}_{ftl}$	none	0.41%	1.06%
$\mathcal{L}_{ntl} + \mathcal{L}_{mtl} + \mathcal{L}_{ftl}$	$\mathcal{L}_{ntl} + \mathcal{L}_{mtl}$	1.01%	1.03%

Selection of positive actions: As introduced in Section 3, the objective of our training losses is to predict positively engaged items in the future. In such a framework, the definition of positive action could affect the quality of the pretrained model. In Table 4, we ablate positive action types in pretraining, evaluated by metrics on downstream HF ranking model. As can be seen in the table, adding Download or Clickthrough action as positive action can both individually provide positive impact, and adding all the actions but Hide and Clickthrough can lead to significant gain on Hide metric. However, adding all actions but Hide will hurt Save metric. The results reflect that selecting positive actions optimally is a non-trivial problem.

Iterations in pretraining: We ablate how the number of iterations during pretraining impact the performance of HF ranking and summarize the results in Figure 3. We test from 0 iteration to 640k iterations, with 0 iteration (no pretraining) as baseline. We observe that as we pretrain longer, the Save and Hide both get better, though not strictly monotonically. Note that the epoch boundaries are marked by yellow vertical lines. We chose to pretrain around 3 epochs to balance pretraining cost and performance. Another interesting observation is the pretraining does not suffer from one epoch overfitting issue which is often observed in training id-based ranking models.

Table 4: Percentage change in HIT@3 when changing pretraining actions, with using Save as baseline.

Action types	Save (lift)	Hide (lift)
Save	0.0%	0.0%
Save + Download	0.21%	-1.63%
Save + Clickthrough	0.02%	-2.67%
All - Hide	-0.2%	-1.76%
All - Hide - Clickthrough	0.19%	-4.1%

Importance of fine-tuning: In Table 5, we ablate the importance of PinFM fine-tuning in HF ranking. In LLMs, next token prediction-based pretraining helps the model memorize world knowledge, but doesn't work out of the box for downstream applications. Fine-tuning is needed for capabilities like instruction following.

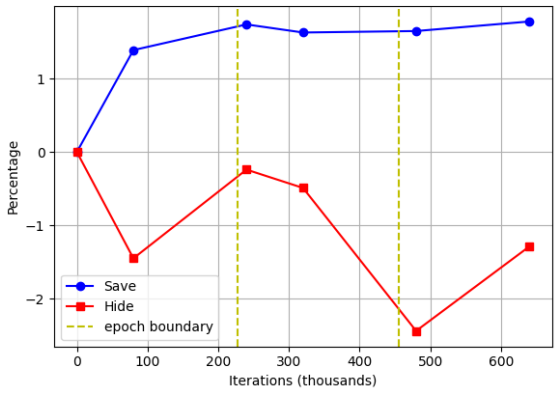


Figure 3: Save and Hide vs. number of pretraining iterations

Similarly, we want to understand if fine-tuning is as important in our foundation model. Experiment results show that without fine-tuning, the lift on Save diminishes to almost none, and Hide is even negatively impacted. This demonstrates that fine-tuning plays a critical role. And this shows the necessity of applying fine-tuning in both HF and I2I surfaces.

Table 5: Percentage change in HIT@3 on HF when adding the PinFM w/ and w/o fine-tuning, with no PinFM as baseline

fine-tuning setting	Save (lift)	Hide (lift)
w/o PinFM	0.0%	0.0%
w/o fine-tuning (freeze PinFM)	0.10%	2.56%
w/ fine-tuning	3.76%	-2.77%

Choice of vocabulary size: In Table 6, we ablate the effect of scaling the vocabulary size. We observed a steady increase in model performance as we scale the embedding vocabulary size from 20 million rows to 160 million rows.

Table 6: Percentage change in Save HIT@3 on HF ranking model at different vocabulary sizes, with 20M as baseline.

Embedding vocabulary size	Save (lift)	Hide (lift)
20M	0.0%	0.0 %
40M	0.81%	-0.34%
80M	0.91%	0.31%
160M	1.98%	0.92%

5.2 Online results

Table 7: Online improvements from A/B test for PinFM. All results are statistically significant with a 95% confidence level.

Metric	HF Ranking	I2I Ranking
Sitewide Saves	**+1.20%**	**+0.72%**
Surface Saves	**+2.60%**	**+2.09%**
Fresh Saves	**+5.70%**	**-0.82%**

We added PinFM to both Home Feed ranking and I2I ranking models and conducted A/B tests against the existing ranking models (Table 7). Although the baselines utilize transformers to encode user sequences [26], the number of parameters in the transformer — combining both the neural network layers and the embedding table — is significantly smaller, comprising less than 0.2% parameters compared to PinFM.

Note that PinFM in I2I was without the cold start remediation techniques in Table 2, because I2I's fresh Save metric drop was within an acceptable range. On the other hand, PinFM in HF showed about -5% Fresh Saves and thus we added the techniques in Table 2, leading to about +10% Fresh Saves lift.

In addition, we observed increase in feed diversity for both HF and I2I which is measured by the number of engaged items from topics that a users have not recently impressed or engaged with. This demonstrated the effectiveness of PinFM to boost both engagement and feed diversity.

6 Conclusion

We presented PinFM, a billion-scale visual discovery platform's first foundational model for user activity sequences. We pretrain a large transformer with over 20 billion parameters using years of user activity data. Subsequently, we fine-tune PinFM with downstream ranking models for each application. During the fine-tuning stage, PinFM learns the interactions between user activity sequences and candidate items while retaining the knowledge acquired during pretraining. We enhanced PinFM's training and serving efficiency by introducing the Deduplicated Cross-Attention Transformer and quantizing the large embedding tables. As a result, we successfully deployed PinFM across multiple applications without incurring significant cost increases. For future work, we aim to further improve the training and serving efficiency of PinFM and incorporate more diverse entities into the input sequences. As the platform's first foundational model, PinFM establishes a new paradigm for the company's recommender systems.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[2] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. TWIN: TWo-stage Interest Network for Lifelong User Behavior Modeling in CTR Prediction at Kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 3785–3794. doi:10.1145/3580305.3599922

[3] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. M6-Rec: Generative Pretrained Language Models are Open-Ended Recommender Systems. *CoRR* abs/2205.08084 (2022). doi:10.48550/ARXIV.2205.08084 arXiv:2205.08084

[4] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] https://arxiv.org/abs/2307.08691

[5] Hamed Firooz, Maziar Sanjabi, Adrian Englhardt, Aman Gupta, Ben Levine, Dre Olgiati, Gungor Polatkan, Iuliia Melnychuk, Karthik Ramgopal, Kirill Talanine, Kutta Srinivasan, Luke Simon, Natesh Sivasubramoniapillai, Necip Fazil Ayan, Qingquan Song, Samira Sriram, Souvik Ghosh, Tao Song, Vignesh Kothapalli, Xiaoling Zhai, Ya Xu, Yu Wang, and Yun Dai. 2025. 360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation. *CoRR* abs/2501.16450 (2025). doi:10.48550/ARXIV.2501.16450 arXiv:2501.16450

[6] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1511.06939

[7] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 585–593. doi:10.1145/3534678.3539381

[8] Chengkai Huang, Tong Yu, Kaige Xie, Shuai Zhang, Lina Yao, and Julian J. McAuley. 2024. Foundation Models for Recommender Systems: A Survey and New Perspectives. *CoRR* abs/2402.11143 (2024). doi:10.48550/ARXIV.2402.11143 arXiv:2402.11143

[9] Lei Huang, Hao Guo, Linzhi Peng, Long Zhang, Xiaoteng Wang, Daoyuan Wang, Shichao Wang, Jinpeng Wang, Lei Wang, and Sheng Chen. 2025. SessionRec: Next Session Prediction Paradigm For Generative Sequential Recommendation. arXiv:2502.10157 [cs.IR] https://arxiv.org/abs/2502.10157

[10] Dmytro Ivchenko, Dennis Van Der Staay, Colin Taylor, Xing Liu, Will Feng, Rahul Kindi, Anirudh Sudarshan, and Shahin Sefati. 2022. TorchRec: a PyTorch Domain Library for Recommendation Systems. In *Proceedings of the 16th ACM Conference on Recommender Systems* (Seattle, WA, USA) *(RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 482–483. doi:10.1145/3523227.3547387

[11] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17-20, 2018*. IEEE Computer Society, 197–206. doi:10.1109/ICDM.2018.00035

[12] Nikhil Khani, Li Wei, Aniruddh Nath, Shawn Andrews, Shuo Yang, Yang Liu, Pendo Abbo, Maciej Kula, Jarrod Kahn, Zhe Zhao, Lichan Hong, and Ed H. Chi. 2024. Bridging the Gap: Unpacking the Hidden Challenges in Knowledge Distillation for Online Ranking Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems, RecSys 2024, Bari, Italy, October 14-18, 2024*, Tommaso Di Noia, Pasquale Lops, Thorsten Joachims, Katrien Verbert, Pablo Castells, Zhenhua Dong, and Ben London (Eds.). ACM, 758–761. doi:10.1145/3640457.3688055

[13] Daya Khudia, Jianyu Huang, Protonu Basu, Summer Deng, Haixin Liu, Jongsoo Park, and Mikhail Smelyanskiy. 2021. FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference. *arXiv preprint arXiv:2101.05615* (2021).

[14] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian J. McAuley. 2023. Text Is All You Need: Learning Language Representations for Sequential Recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 1258–1267. doi:10.1145/3580305.3599519

[15] Mingfu Liang, Xi Liu, Rong Jin, Boyang Liu, Qiuling Suo, Qinghai Zhou, Song Zhou, Laming Chen, Hua Zheng, Zhiyuan Li, Shali Jiang, Jiyan Yang, Xiaozhen Xia, Fan Yang, Yasmine Badr, Ellie Wen, Shuyu Xu, Hansey Chen, Zhengyu Zhang, Jade Nie, Chunzhi Yang, Zhichen Zeng, Weilin Zhang, Xingliang Huang, Qianru Li, Shiquan Wang, Evelyn Lyu, Wenjing Lu, Rui Zhang, Wenjun Wang, Jason Rudy, Mengyue Hang, Kai Wang, Yinbin Ma, Shuaiwen Wang, Sihan Zeng, Tongyi Tang, Xiaohan Wei, Longhao Jin, Jamey Zhang, Marcus Chen, Jiayi Zhang, Angie Huang, Chi Zhang, Zhengli Zhao, Jared Yang, Qiang Jin, Xian Chen, Amit Anand Amlesahwaram, Lexi Song, Liang Luo, Yuchen Hao, Nan Xiao, Yavuz Yetim, Luoshang Pan, Gaoxiang Liu, Yuxi Hu, Yuzhen Huang, Jackie Xu, Rich Zhu, Xin Zhang, Yiqun Liu, Hang Yin, Yuxin Chen, Buyun Zhang, Xiaoyi Liu, Xingyuan Wang, Wenguang Mao, Zhijing Li, Qin Huang, Chonglin Sun,

Shupin Mao, Benjamin Au, Jingzheng Qin, Peggy Yao, Jae-Woo Choi, Bin Gao, Ernest Wang, Lei Zhang, Wen-Yen Chen, Ted Lee, Jay Zha, Yi Meng, Alex Gong, Edison Gao, Alireza Vahdatpour, Yiping Han, Yantao Yao, Toshinari Kureha, Shuo Chang, Musharaf Sultan, John Bocharov, Sagar Chordia, Xiaorui Gan, Peng Sun, Rocky Liu, Bo Long, Wenlin Chen, Santanu Kolay, and Huayu Li. 2025. External Large Foundation Model: How to Efficiently Serve Trillions of Parameters for Online Ads Recommendation. *CoRR* abs/2502.17494 (2025). doi:10.48550/ARXIV.2502.17494 arXiv:2502.17494

[16] Zihan Liu, Yupeng Hou, and Julian J. McAuley. 2024. Multi-Behavior Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*, Edoardo Serra and Francesca Spezzano (Eds.). ACM, 1575–1585. doi:10.1145/3627673.3679730

[17] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *CoRR* abs/1906.00091 (2019). arXiv:1906.00091 http://arxiv.org/abs/1906.00091

[18] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[19] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. PinnerFormer: Sequence Modeling for User Representation at Pinterest. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 3702–3712. doi:10.1145/3534678.3539156

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[21] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Mahesh Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/20dcab0f14046a5c6b02b61da9f13229-Abstract-Conference.html

[22] Steffen Rendle. 2010. Factorization Machines. In *2010 IEEE International Conference on Data Mining*. 995–1000. doi:10.1109/ICDM.2010.127

[23] Zihua Si, Lin Guan, Zhongxiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, Kai Zheng, Chenbin Zhang, Yanan Niu, Yang Song, and Kun Gai. 2024. TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling for Enhanced CTR Prediction at Kuaishou. *CoRR* abs/2407.16357 (2024). doi:10.48550/ARXIV.2407.16357 arXiv:2407.16357

[24] Ruoxi Wang, Rakesh Shivanna, Derek Zhiyuan Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, Jure Leskovec, Marko Grobelnik, Marc Najork, Jie Tang, and Leila Zia (Eds.). ACM / IW3C2, 1785–1797. doi:10.1145/3442381.3450078

[25] Chuhan Wu, Fangzhao Wu, Tao Qi, Jianxun Lian, Yongfeng Huang, and Xing Xie. 2020. PTUM: Pre-training User Model from Unlabeled User Behaviors via Self-supervision. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1939–1944. doi:10.18653/V1/2020.FINDINGS-EMNLP.174

[26] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 5249–5259. doi:10.1145/3580305.3599918

[27] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.

[28] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International conference on machine learning*. PMLR, 10524–10533.

[29] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, Yike Guo and Faisal Farooq (Eds.). ACM, 974–983. doi:10.1145/3219819.3219890

[30] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. https://openreview.net/forum?id=xye7iNsgXn

[31] Gaowei Zhang, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Scaling Law of Large Sequential Recommendation Models. In *Proceedings of the 18th ACM Conference on Recommender Systems, RecSys 2024, Bari, Italy, October 14-18, 2024*, Tommaso Di Noia, Pasquale Lops, Thorsten Joachims, Katrien Verbert, Pablo Castells, Zhenhua Dong, and Ben London (Eds.). ACM, 444–453. doi:10.1145/3640457.3688129

[32] Guorui Zhou, Xiaoqiang Zhu, Chengru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, Yike Guo and Faisal Farooq (Eds.). ACM, 1059–1068. doi:10.1145/3219819.3219823